

READ THE DIFF FIRST

SPC **h** **d**

THIS FILE VS INDEX

side-by-side vim diff — what have I changed since staging?

SPC **h** **D**

VS LAST COMMIT

everything since HEAD, staged or not

]c / **[c**

HOP CHANGE TO CHANGE

same keys inside diff view and in a normal buffer

do

TAKE THE OTHER SIDE

diff-obtain: pull their version of this change into your buffer

dp

PUSH YOURS ACROSS

diff-put: send your version to the other window

:diffoff!

DONE COMPARING

or just close the other window

SPC **h** **p**

QUICK PEEK INSTEAD

inline hunk popup when a full split is overkill

SPC **g** **e**

ALL CHANGED FILES

git status in the sidebar — your review checklist

nvim -d a b

DIFF ANY TWO FILES

or :vert difffsplit other_file from inside a buffer

STAGE LIKE A SURGEON

]c

SPC h p

SPC h s

V ... SPC hs

SPC h r

SPC h u

SPC g e

SPC gg → c

result

1 · WALK TO A HUNK

start at the top of the file, visit every change

2 · LOOK AT IT

does this belong in the commit you're building?

3 · YES → STAGE IT

3½ · ONLY THESE LINES

select part of a hunk — debug print and real fix in one hunk? split them

DEBRIS → DISCARD

stray prints, commented experiments — gone before they embarrass you

OOPS → UN-STAGE

staged the wrong hunk? toggle it back out

REPEAT PER FILE

the git-status sidebar is your worklist across the repo

COMMIT WHAT'S STAGED

lazygit shows exactly what's going in — no surprises

COMMITMENTS THAT REVIEW THEMSELVES

one idea per commit — reviewers and future-you both win

REWRITE HISTORY SAFELY

SPC **gg** → **4**

s / **f**

r

e

d

CTRL **j** / **k**

↑C ... **↑V**

3 → **r** / **M**

z

GO TO THE COMMITS PANEL

everything below happens there — before you push

SQUASH / FIXUP DOWN

merge a "typo fix" commit into the real one below it — f discards its message

REWORD THE MESSAGE

works on any unpushed commit, not just the last

EDIT — STOP THE REBASE HERE

amend files into an older commit, then continue

DROP A COMMIT

delete it from history entirely

MOVE A COMMIT UP / DOWN

reorder history by just... moving it

CHERRY-PICK

copy commits here, paste them on another branch

REBASE ONTO / MERGE IN

branches panel: rebase your branch onto the highlighted one, or merge it into yours

UNDO THE LAST ACTION

lazygit tracks its own undo via reflog — experiment bravely

REWRITE BEFORE PUSH, NEVER AFTER

IT'S ALL INTERACTIVE REBASE UNDERNEATH

CONFLICTS, CALMLY



CONFLICTS SHOW IN RED

the files panel marks every conflicted file



OPEN THE CONFLICT VIEW

each conflict shown as competing hunks



PICK THE HIGHLIGHTED SIDE

arrow keys move between ours / theirs and between conflicts



KEEP BOTH

when the two changes don't actually collide



IT OFFERS TO CONTINUE

resolve the final conflict and lazygit asks to continue the merge / rebase



ABORT · SKIP · CONTINUE

mid-rebase options menu — abort is always available, nothing is ruined



OR FIX BY HAND IN NVIM

search the markers, keep the lines you want, delete the rest + all three markers



NOTHING IS EVER LOST

every state you've been in is still reachable — worst case, reset to the reflog entry

GIT AT THE PROMPT

gst

gd

/

gdca

ga

/

gaa

gcmmsg

"..."

gco

/

gcb

gpc

/

glc

gsta

/

gstp

gll

git-clean-branches

STATUS

oh-my-zsh git plugin — you have all of these already

DIFF / DIFF STAGED

gdca answers "what exactly am I about to commit?"

ADD / ADD ALL

prefer hunk staging in nvim — gaa is for the rare all-in commit

COMMIT WITH MESSAGE

CHECKOUT / NEW BRANCH

PUSH / PULL THIS BRANCH

your functions — gpc sets upstream on a brand-new branch automatically

STASH / STASH POP

park changes, switch context, get them back

THE PRETTY GRAPH

your alias — all branches, decorated, relative dates

DELETE MERGED BRANCHES

your function — keeps main/master/dev, sweeps the rest

WHY DOES THIS CODE EXIST?

SPC **h** **b**

BLAME THIS LINE

commit, author, date, full message — the "why" is in the message

SPC **t** **b**

BLAME AS YOU BROWSE

inline blame on every line you visit — leave it on while exploring

SPC **g** **L**

THIS FILE'S WHOLE LIFE

lazygit log scoped to the current file — capital L

SPC **g** **l**

REPO HISTORY

enter on any commit to browse the files it touched

gll

SEE THE BRANCH SHAPE

your graph alias — where did this branch diverge?

`git log -S "needle"`

WHEN WAS THIS ADDED?

the pickaxe — finds commits that added or removed that exact string

`git log -L 10,25:f.go`

HISTORY OF THESE LINES

every change ever made to lines 10-25 of that file

SPC **g** **o** **b**

SHARE THE EVIDENCE

visual-select the lines, get a github permalink